SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

---

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Answer

Consider the following SQL query:

Answer

1. Problem Understanding

Query optimization is the process of selecting an **efficient execution strategy** for an SQL query. A complex SQL query may contain selections, projections, joins, nested queries, and sub-queries.

The main objective is to reduce:

- Disk I/O operations
- CPU processing
- Memory usage
- Intermediate relation size
- Overall query execution time

Heuristic optimization applies predefined rules to transform a query into an equivalent but more efficient form.

2. Example Query

Consider the following SQL query:

```
mysql> SELECT E.name, D.dept_name
    -> FROM Employee E
    -> JOIN Department D
    -> ON E.dept_id = D.dept_id
    -> WHERE E.salary > 50000
    -> AND E.dept_id IN
    -> (
    ->     SELECT dept_id
    ->     FROM Department
    ->     WHERE location = 'Chennai'
    -> );
```

The query contains:

- A join between Employee and Department
- A selection condition salary > 50000
- A nested sub-query
- A selection condition location = 'Chennai'
- A final projection of name and dept_name

3. Initial Query Analysis

The query can be represented using relational algebra.

The main operation is:

where:

$$\pi_{E.name,D.dept\_name}(\sigma_{E.salary>50000 \wedge E.dept\_id \in S}(Employee \bowtie Department))$$

This initial form may generate large intermediate relations.

4. Heuristic Rule 1 – Push Selection Down

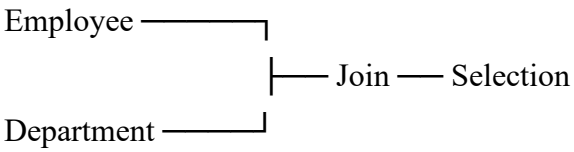
Selection operations should be performed **as early as possible**.

Instead of joining all Employee records and then checking salary, filter Employee first:

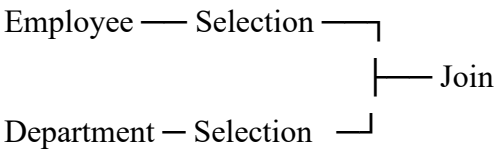
Similarly, filter Department:

This reduces the number of tuples participating in later operations.

Before



After



5. Heuristic Rule 2 – Simplify the Sub-query

The sub-query is:

```
mysql> SELECT dept_id
    -> FROM Department
    -> WHERE location = 'Chennai';
```

Its relational algebra representation is:

Only the required dept_id is retained.

Therefore, unnecessary Department attributes are removed before further processing.

6. Heuristic Rule 3 – Push Projection Down

The final query only requires:

From Employee:

- name
- dept_id

From Department:

- dept_id
- dept_name

Therefore:

$$E'' = \pi_{name, dept\_id}(\sigma_{salary > 50000}(Employee))$$

And

$$D'' = \pi_{dept\_id, dept\_name}(\sigma_{location = 'Chennai'}(Department))$$

This reduces the number of columns stored in intermediate relations.

7. Heuristic Rule 4 – Perform Selective Operations First

The most restrictive conditions should be applied early.

For example:

salary > 50000

location = 'Chennai'

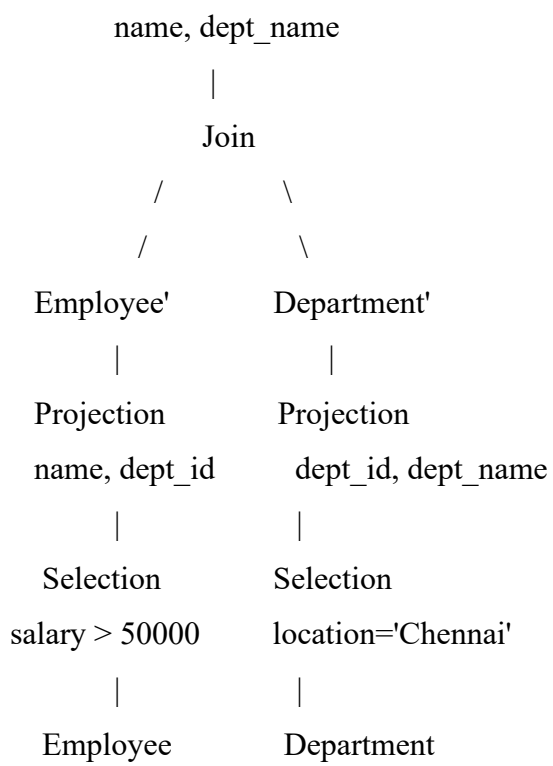
are applied before the join.

This means fewer records enter the join operation.

8. Optimized Execution Plan

The optimized plan can be represented as:

Projection



The optimizer therefore performs:

1. Selection on Employee.
2. Selection on Department.
3. Projection on required attributes.
4. Join the reduced relations.
5. Perform final projection.

9. Heuristic Rules Applied

Rule	Application
Selection pushdown	Filter tuples before joins
Projection pushdown	Remove unnecessary attributes
Sub-query simplification	Convert sub-query into relational operations
Selective operation first	Apply restrictive conditions early

10. Advantages

The optimized plan:

- Reduces intermediate relation size.
- Reduces disk access.
- Requires less memory.
- Reduces CPU processing.
- Makes join operations faster.
- Improves overall query response time.

Conclusion

Heuristic optimization transforms the original query into an equivalent execution plan with smaller intermediate relations. By pushing selections and projections toward the base relations and performing selective operations early, the query can be executed more efficiently.

Optimized Query → Less I/O → Less Processing → Faster Execution

Q2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join.

Answer

1. Problem Understanding

Cost-based query optimization estimates the execution cost of different query execution plans and selects the plan having the **lowest estimated cost**.

Join algorithms are important because joins can involve a large number of tuples and disk pages.

The three commonly used join techniques are:

1. Nested-loop join
2. Sort-merge join
3. Hash join

2. Given Example

Consider two relations:

Relation	Number of Tuples	Number of Pages
R	1,000	100
S	10,000	1,000

Assume an **equality join** between R and S.

3. Nested-Loop Join

Nested-loop join compares tuples/pages from one relation with the other relation.

For a simple page-oriented nested-loop join:

where:

$$Cost = B(R) + B(R) \times B(S)$$

- B(R)=100
- B(S)=1000

Therefore:

$$Cost = 100 + (100 \times 1000)$$

$$Cost = 100 + 100000$$

$$Cost = 100100\ I/O$$

This is expensive because the inner relation can be scanned repeatedly.

4. Sort-Merge Join

Sort-merge join performs the following operations:

Step 1

Sort relation R on the join attribute.

Step 2

Sort relation S on the join attribute.

Step 3

Merge the two sorted relations.

General cost:

For the given relations:

Additional cost is incurred for sorting.

Therefore, sort-merge join can be effective when:

- Relations are already sorted.
- Sorting is required for another operation.
- The join is not suitable for hashing.

5. Hash Join

Hash join is especially effective for an equality join.

A hash function divides both relations into corresponding partitions.

Approximate cost:

$$CostHash=3(B(R)+B(S))$$

Therefore:

$$=3(100+1000)$$

$$=3(1100)$$

$$=3300\ I/O$$

6. Comparison

Algorithm	Estimated Cost	Observation
Nested Loop	100100 I/O	Very expensive
Sort-Merge	Depends on sorting	Moderate
Hash Join	≈ 3300 I/O	Lowest for equality join

7. Selection

Since the query is an **equality join**, hash join is highly suitable.

Therefore:

Select Hash Join

8. Justification

Hash join is selected because:

- The join condition is equality-based.
- It avoids repeated scanning.
- It does not require complete sorting.
- Its estimated I/O cost is much lower.
- It performs well for large relations.

Conclusion

Cost-based optimization compares different execution strategies and selects the least expensive one. For the given example, **Hash Join** provides the most efficient execution plan.

Hash Join is the preferred method

Q3. Implement JDBC connectivity in Java to a MySQL database and execute parameterized queries using PreparedStatement.

Answer

1. Problem Understanding

JDBC stands for **Java Database Connectivity**.

It is a standard Java API that allows Java applications to communicate with relational databases such as MySQL.

The JDBC process consists of:

Java Application
↓
JDBC API
↓
JDBC Driver
↓
MySQL Database

2. Requirements

To establish JDBC connectivity:

- Java JDK
- MySQL Server
- MySQL database
- MySQL JDBC Driver
- Database username and password

3. Database Example

Assume a database named college contains:

```
mysql> CREATE DATABASE college;
Query OK, 1 row affected (0.03 sec)
```

and a table:

```
mysql> Use College;
Database changed
mysql> CREATE TABLE Student(
    ->     id INT PRIMARY KEY,
    ->     name VARCHAR(50),
    ->     dept VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

4. JDBC Program

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/college";

        String user = "root";
        String password = "root";
```

```
String sql =
   "SELECT * FROM Student WHERE dept = ?";

try {

   Connection con =
      DriverManager.getConnection(
         url, user, password
      );

   PreparedStatement ps =
      con.prepareStatement(sql);

   ps.setString(1, "CSE");

   ResultSet rs =
      ps.executeQuery();

   while (rs.next()) {

      System.out.println(
         rs.getInt("id") + " " +
         rs.getString("name") + " " +
         rs.getString("dept")
      );
   }

   rs.close();
   ps.close();
   con.close();

}
catch (SQLException e) {
```

```
            System.out.println(
               "Database Error: " +
               e.getMessage()
            );
        }
    }
}
```

---

5. Explanation

Connection

```
Connection con =
DriverManager.getConnection(url, user, password);
```

This establishes a connection between Java and MySQL.

Prepared Statement

```
PreparedStatement ps =
con.prepareStatement(sql);
```

It prepares the SQL query.

Parameter

```
ps.setString(1, "CSE");
```

The? placeholder is replaced with "CSE".

Execution

```
ResultSet rs = ps.executeQuery();
```

This executes the SELECT query.

6. Advantages of PreparedStatement

PreparedStatement is better than directly constructing SQL strings because:

1. It supports parameterized queries.
2. It helps prevent SQL injection.
3. It improves code readability.
4. It supports different parameter data types.
5. Prepared queries can be reused.
6. It provides better security.

Conclusion

JDBC provides connectivity between Java and MySQL, while PreparedStatement enables secure and efficient parameterized SQL execution.

Q4. Determine whether a schedule of four concurrent transactions is conflict-serializable using a precedence graph.

Answer

1. Problem Understanding

- When multiple transactions execute concurrently, their operations may be interleaved.
- A schedule is **conflict-serializable** if it is conflict-equivalent to some serial schedule.
- A **precedence graph** is used to determine conflict serializability.

2. Rules for Conflicting Operations

Two operations conflict when:

1. They belong to different transactions.
2. They operate on the same data item.
3. At least one operation is a write.

.The conflicting pairs are:

- Read-Write
- Write-Read
- Write-Write

Read-Read does not conflict.

3. Example Schedule

Consider:

Step	Operation
1	T1: R(X)
2	T2: W(X)
3	T3: R(Y)
4	T4: W(Y)
5	T1: W(X)
6	T3: W(Y)

4. Construct Precedence Graph

For X:

T1 reads X before T2 writes X.

Therefore:

T2 writes X before T1 writes X.

Therefore:

$$T1 \rightarrow T2$$

For Y:

T3 reads Y before T4 writes Y.

Therefore:

T4 writes Y before T3 writes Y.

Therefore:

$$T2 \rightarrow T1$$

5. Graph Analysis

The graph contains:

$$T1 \rightarrow T2 \rightarrow T1$$

This forms a cycle.

Similarly:

$$T3 \rightarrow T4 \rightarrow T3$$

also forms a cycle.

6. Result

A precedence graph containing a cycle indicates that the schedule is **not conflict-serializable**.

Schedule is NOT conflict-serializable

Important Rule

Acyclic Graph → Conflict Serializable

Cyclic Graph → Not Conflict Serializable

Conclusion

The precedence graph is used to analyze dependencies between transactions. Since the example graph contains cycles, no equivalent serial schedule exists.

Q5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve deadlocks.

Answer

1. Introduction

Two-Phase Locking (2PL) is a concurrency-control protocol used in DBMS to ensure **conflict serializability**.

A transaction operates in two phases:

Phase 1 – Growing Phase

The transaction can:

- Acquire locks.
- Upgrade locks.

It cannot release locks.

Phase 2 – Shrinking Phase

The transaction can:

- Release locks.

It cannot acquire new locks.

2. Types of Locks

Shared Lock – S

Used for reading.

Multiple transactions can simultaneously hold a shared lock.

Exclusive Lock – X

Used for writing.

Only one transaction can hold an exclusive lock on a data item.

3. Example

Consider:

T1:

Lock-X(A)

Write(A)

Lock-X(B)

Write(B)

Unlock(A)

Unlock(B)

and:

T2:

Lock-X(B)

Write(B)

Lock-X(A)

Write(A)

Unlock(B)

Unlock(A)

4. Deadlock Formation

Suppose T1 first obtains:

X(A)

and T2 obtains:

X(B)

Now:

T1 requests X(B)

But B is held by T2.

Therefore:

$$T1 \rightarrow T2$$

Similarly:

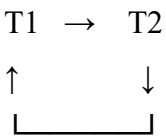
T2 requests X(A)

But A is held by T1.

Therefore:

$$T2 \rightarrow T1$$

The wait-for graph becomes:



There is a cycle.

Therefore:

Deadlock occurs

5. Deadlock Resolution

One transaction must be selected as the victim.

For example:

ROLLBACK(T2)

When T2 is rolled back:

- T2 releases B.
- T1 obtains B.
- T1 completes.
- T2 can be restarted later.

6. Important Property

Basic 2PL guarantees:

Conflict Serializability

However:

Basic 2PL does NOT prevent deadlocks

Deadlocks can still occur when transactions hold locks and wait for locks held by each other.

Conclusion

Two-phase locking provides serializable execution by separating lock acquisition and lock release into two phases. Deadlocks can be detected using a wait-for graph and resolved by aborting one of the participating transactions.

Q6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes.

Answer

1. Introduction

- Deadlocks occur when transactions wait for one another indefinitely.
- Timestamp-based schemes prevent deadlocks by assigning a unique timestamp to every transaction.

Assume:

Transaction	Timestamp
T1	5
T2	10

A smaller timestamp indicates an **older transaction**.

Therefore:

$$T1 > age T2$$

That means T1 is older and T2 is younger.

A. Wait-Die Scheme

Rule

In Wait-Die:

- Older transaction requests a lock held by younger → **Wait**
- Younger transaction requests a lock held by older → **Die/Rollback**

Case 1: T1 requests T2's lock

T1 is older.

Therefore:

Case 2: T2 requests T1's lock

T2 is younger.

Therefore:

T2 dies / rolls back

B. Wound-Wait Scheme

Rule

In Wound-Wait:

- Older transaction requests lock held by younger → **Abort/Wound younger**
- Younger transaction requests lock held by older → **Wait**

Case 1: T1 requests T2's lock

T1 is older.

Therefore:

T1 wounds T2

T2 is rolled back.

Case 2: T2 requests T1's lock

T2 is younger.

Therefore:

T2 waits

4. Comparison

Condition	Wait-Die	Wound-Wait
Older → Younger	Wait	Abort younger
Younger → Older	Abort younger	Wait
Deadlock	Prevented	Prevented
Nature	Non-preemptive	Preemptive

5. Why They Prevent Deadlocks

- Both protocols enforce transaction ordering based on timestamps.
- A circular waiting condition cannot continue indefinitely because transactions are either forced to wait according to age or rolled back.

Conclusion

Wait-Die and Wound-Wait are timestamp-based deadlock prevention techniques.

The easiest way to remember:

Wait-Die → Old waits, Young dies

Wound-Wait → Old wounds, Young waits

Q7. Given transaction logs with checkpoints, apply the Immediate Update recovery protocol after a system failure.

Answer

1. Introduction

Immediate Update is a recovery technique in which database modifications may be written to disk **before the transaction commits**.

Therefore, after a system failure, the recovery manager must determine:

- Which transactions committed?
- Which transactions were active?
- Which changes need to be redone?
- Which changes need to be undone?

The main recovery operations are:

UNDO + REDO

2. Example Log

Consider:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<CHECKPOINT>

<START T3>
<T3, C, 300, 350>

<COMMIT T3>

SYSTEM FAILURE

3. Identify Transaction Status

T1

START T1

...

COMMIT T1

Therefore:

T1 = Committed

T2

T2 has a START record but no COMMIT.

Therefore:

T2 = Uncommitted

T3

T3 has:

COMMIT T3

Therefore:

T3 = Committed

4. REDO Operation

Committed transactions must be redone to ensure their changes are present.

Therefore:

REDO(T1)

and:

REDO(T3)

So:

A = 150

C = 350

5. UNDO Operation

T2 did not commit.

Therefore, its update must be removed.

Original value:

B = 200

Updated value:

B=250

UNDO restores:

B=200

6. Recovery Table

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Committed	REDO

7. Role of Checkpoint

- The checkpoint provides a known recovery point.
- Instead of examining the entire log from the beginning, the recovery system can use the checkpoint to reduce the amount of log processing required.

Final Database State

A = 150

B = 200

C = 350

Conclusion

Immediate Update recovery ensures that:

- Committed changes are retained.
- Uncommitted changes are removed.
- Database consistency is restored.

Immediate Update = UNDO + REDO

Q8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file.

Answer

1. Introduction

Deferred Update is a recovery technique in which modifications made by a transaction are not written to the database until the transaction commits.

Therefore, uncommitted changes cannot exist in the database.

This makes recovery simpler.

Deferred Update = NO-UNDO + REDO

2. Example Log

Consider:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<START T3>
<T3, C, 300, 350>

SYSTEM FAILURE

3. Transaction Status

T1

T1 has a COMMIT record.

Therefore:

T1 = Committed

T2

No COMMIT exists.

Therefore:

T2=Uncommitted

T3

No COMMIT exists.

Therefore:

T3=Uncommitted

4. REDO Committed Transactions

T1 committed.

Therefore:

REDO(T1)

The value of A becomes:

A=150

5. Ignore Uncommitted Transactions

T2 and T3 were not committed.

Because deferred update does not write their changes to the database before commit:

No UNDO is required

T2 and T3 are simply ignored during recovery.

6. Recovery Table

Transaction	Status	Recovery
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Uncommitted	Ignore

7. Final State

The changes belonging to T2 and T3 are not applied.

Comparison

Recovery Method	UNDO	REDO
Immediate Update	Yes	Yes
Deferred Update	No	Yes

Conclusion

Deferred Update simplifies recovery because uncommitted transactions never modify the database on disk.

NO-UNDO/REDO

Q9. Design a concurrency control mechanism using Strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Answer

1. Problem Understanding

A banking system requires strong consistency because incorrect concurrency handling may produce incorrect account balances.

Suppose:

- Account A = ₹10,000
- Account B = ₹5,000
- T1 transfers ₹2,000 from A to B.
- T2 reads the balance while T1 is executing.

The total balance must always remain:

2. Transaction T1

T1 performs:

$$A=A-2000$$

and:

$$B=B+2000$$

Final values should be:

$$A=₹8000$$

$$B=₹7000$$

3. Strict 2PL

Strict 2PL requires a transaction to hold its exclusive locks until it commits or aborts.

T1:

LOCK-X(A)

LOCK-X(B)

A = A - 2000

B = B + 2000

COMMIT

UNLOCK(A)

UNLOCK(B)

4. Transaction T2

T2 only reads.

Therefore, it requires shared locks:

LOCK-S(A)

LOCK-S(B)

READ(A)

READ(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

5. Concurrent Execution

Suppose T1 starts first.

T1: LOCK-X(A)

T1: LOCK-X(B)

T1: A = A - 2000

T1: B = B + 2000

T1: COMMIT

T1: UNLOCK(A)

T1: UNLOCK(B)

T2: LOCK-S(A)

T2: LOCK-S(B)

T2: READ(A)

T2: READ(B)

T2: COMMIT

T2 therefore reads:

$$A=₹8000$$

$$B=₹7000$$

Total:

$$₹8000+₹7000=₹15000$$

Thus the total balance remains correct.

6. What Would Happen Without Strict 2PL?

Suppose T2 reads A after T1 has deducted ₹2,000 but before T1 adds ₹2,000 to B.

T2 might observe:

$$A=₹8000$$

$$B=₹5000$$

Total:

$$₹8000+₹5000=₹13000$$

This is an **inconsistent intermediate state**.

Strict 2PL prevents this situation.

7. Benefits

Strict 2PL:

- Prevents dirty reads.
- Prevents dirty writes.
- Prevents cascading rollback.
- Ensures conflict serializability.
- Maintains database consistency.
- Protects financial transactions.

Conclusion

Strict 2PL ensures that T2 cannot read the accounts while T1 has partially completed the transfer. T2 waits until T1 commits and releases its exclusive locks.

Therefore, the banking system maintains:

Consistency + Isolation + Serializability

Q10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Answer

1. Problem Understanding

- Query optimization aims to find an efficient execution plan for a query.
- Heuristic optimization uses rules rather than calculating every possible execution cost.

The three important heuristics required are:

1. Push selection down.
2. Perform projection early.
3. Order joins to produce smaller intermediate results.

2. Pseudocode

Algorithm Heuristic_Query_Optimization(Query)

Input:

SQL Query

Output:

Optimized Query Execution Plan

BEGIN

Step 1:

Parse the SQL query.

Step 2:

Convert the SQL query into a relational algebra expression.

Step 3:

Construct the initial query tree.

Step 4:

Identify all selection conditions.

Step 5:

Push each selection operation as close as possible to the corresponding base relation.

Step 6:

Identify all required attributes.

Step 7:

Push projection operations down toward the base relations.

Step 8:

Remove unnecessary attributes from intermediate relations.

Step 9:

Identify all join operations.

Step 10:

Estimate the size of the intermediate relations.

Step 11:

Select the join order that
produces the smallest
intermediate relation first.

Step 12:
Replace inefficient sub-queries
with equivalent joins when possible.

Step 13:
Generate the optimized
query execution plan.

Step 14:
Return the optimized plan.

END

3. Explanation of Each Heuristic

A. Push Selection Down

Selection filters rows.

For example:

σsalary > 50000 (Employee)

should be performed before a large join whenever possible.

This reduces the number of tuples.

B. Project Early

Projection removes unnecessary columns.

For example:

πname, dept_id (Employee)

only retains required attributes.

This reduces:

- Memory consumption
- Disk I/O
- Intermediate relation size

C. Order Joins Carefully

Suppose:

R1=1000 tuples

R2=100 tuples

R3=10000 tuples

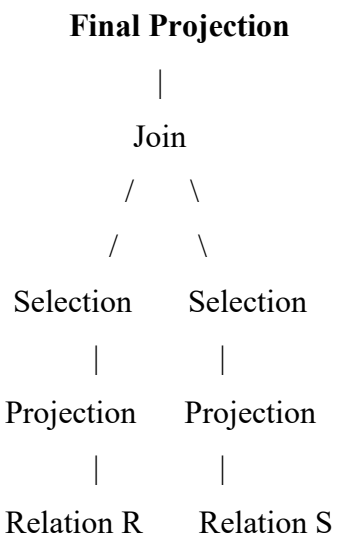
It is usually beneficial to first perform a selective operation involving smaller/intermediate results rather than immediately joining the largest relations.

The goal is:

Minimize Intermediate Relation Size

4. Optimized Query Tree

Conceptually:



5. Advantages

The heuristic approach:

- Reduces intermediate data.
- Reduces disk I/O.
- Reduces memory requirements.
- Reduces CPU operations.
- Improves query execution speed.
- Produces a more efficient execution plan.

Conclusion

The pseudocode applies selection pushdown, early projection, and efficient join ordering. These heuristics reduce the size of intermediate relations and improve the overall performance of query execution.

Filter Early + Project Early + Join Efficiently